

Listener Sentry Logs — Investigation Findings (2026-08-10)

Listener Sentry Logs — Investigation Findings (2026-08-10)

Source files

Three data sources:

File	Logged window (UTC)	Servers (containers)	Release
<code>trace_item_full_export_2026-August-10_122171.jsonl</code>	2026-08-09 07:39:32 → 07:54:04	<code>1de92c8e7b38</code> only	<code>cff88c39fed...</code>
<code>trace_item_full_export_2026-August-10_122181.jsonl</code>	2026-08-10 07:37:41 → 07:45:53	<code>a79d7a0282a4</code> (8,574) + <code>1de92c8e7b38</code> (1,426)	<code>cff88c39fed...</code>
<code>Errors 2026-08-10T08_44_34.csv</code> (Sentry issue export, project <code>tscopier-worker-production</code>)	2026-08-10 07:44:37 → 08:44:00	—	—

Filenames say “August-10” (the export/download date). The **in-log timestamps** are the source of truth.

Plain English summary

What was happening

1. The “close part of a trade” feature was stuck retrying forever. Our system sometimes closes part of a position (e.g. take profit on a portion). To do that it tells the broker “close ticket 1278201”. The broker answered **“unknown ticket”** — meaning “I don’t have that position anymore”. But the code only has a list of *recognised* answers (like “not found”, “already closed”, “invalid ticket”). The exact words **“unknown ticket” were not on that list**, so the code did not understand that the position was already gone. Instead of treating it as “fine, nothing left to do — clean up”, it treated it as a temporary hiccup and **retried every ~0.4 seconds for days**. It’s like calling a bank to close an account that was already closed, and the software never realising the account is gone, so it keeps calling over and over. That one trade generated **505 errors in 14.5 minutes**.

2. The audit paper trail was being silently thrown away. Every time the worker closes anything at the broker, it writes an “audit” row so we have a record. But the table it writes to requires a field called `signal_id` that the code never fills in. So **every audit write failed at the database**, and the failure was only logged, not fixed. Result: the close-audit record for **every** broker close in those windows is missing.

3. The broker connection was unstable. The `FxSocket` connection kept dropping (TLS/network disconnects) and returning **empty position snapshots** (it claimed 0 open trades when our system tracked 1–14). Our monitors are deliberately safe: when the broker returns an empty list they *refuse* to mass-close trades, because they suspect a disconnect. That’s the right guard — but combined with problem 1 it means a trade that is genuinely gone at the broker stays marked “open” in our database forever, which feeds the infinite retry loop.

4. Telegram was rate-limited. Telegram rate-limits us with “flood waits” (sleep 24–31s). One user session failed **452 message polls in 8 minutes**, and 3 public channels (`@forex_vipxauusd`, `@gold_pro_forex_trader_xauusd`, `@gold_pro_trader_forex_signals0`) could not be resolved. Message ingestion for that session was severely degraded.

5. The whole copy pipeline was degraded that morning, not just partial-TP. In a single hour (07:44–08:44 UTC Aug 10) Sentry captured 35 business issues: 12 copy failures (mostly **not enough money / margin**, on XAUUSD sells), 10 copies blocked (daily risk limit + pre-send skips), 5 broker order rejections, 4 accounts treated as unavailable (broker session disconnected), 4 management instructions failed. This matches database rows showing `order_send` failures, `SymbolSelect failed`, and risk-limit hits all morning.

Who was affected

From these log exports we can name the **system-level entities** (accounts, trades, channels). The exports only contain UUIDs, not customer names/emails — mapping them to specific customers requires a lookup against the accounts table.

- **Trade ticket** `1278201` (trade `ffcd18b3`), on broker account `edf84514-77c7-4326-93ab-2c6d20a84172` — the main victim. Its two take-profit legs have been stuck “pending, retrying” **since Aug 7** and are still `pending` as of Aug 10.
- **Trade ticket** `1278230` (trade `ef03693f`), on broker account `d72087d3-5c47-4e23-9748-609f1444f116` — same failure until its legs were cancelled on Aug 10.
- **14+ broker accounts** with FxSocket TLS disconnects / empty snapshots (accounts affected by problem 3).
- **Telegram listener user session** `c8a32918` and the 3 channels listed above — this affects whoever that session listens to (degraded ingestion, late/missing signals).
- **Account** `d72087d3` also had `SymbolSelect failed` on GBPUSD/XAUUSD, which blocked new entries.

Bottom line: a handful of accounts were hit hard (stuck partial closes, missing audit, lost signals), and the affected sessions risked missed or late trades that morning.

Volume by severity

Severity	File 1 (Aug 9)	File 2 (Aug 10)
info	6,920	8,017
warn	2,575	1,856
error	505	127
Total	10,000	10,000

Volume by subsystem (message prefix)

Subsystem prefix	File 1	File 2
(no prefix — gramjs/Telegram output)	5,911	6,671
<code>[orderCloseAudit]</code>	1,011	240
<code>[partialTpMonitor]</code>	607	209
<code>[v2ReconcileMonitor]</code>	574	282
<code>[userListener]</code>	532	1,664
<code>[openTradeReconcile]</code>	525	256
<code>[telegram-conn]</code>	399	237
<code>[basketReconcileTargets]</code>	351	203
<code>[fxsocketWsClient]</code>	—	90
<code>[tradeExecutor]</code>	—	69
<code>[trailingStopMonitor]</code>	48	27
<code>[copyLimitMonitor]</code>	29	—
<code>[virtualPendingMonitor]</code>	—	23
<code>[autoManagementMonitor]</code>	8	17
misc	5	15

Failure categories

1. `partialTpMonitor` — `unknown ticket` (file 1: 505 errors; file 2: 119 + 1 TLS)

Every single **error-severity** line in **file 1** is the same failure. File 2's 120 errors are the same shape (119 **unknown ticket** + 1 TLS socket error):

```
[partialTpMonitor] fire failed partial=<uuid> ticket=<NUM>: unknown ticket
```

Heartbeat ticks confirm **5/5 triggered partials fail**:

```
[partialTpMonitor] tick rows=28 groups=6 triggered=5 fired=0_ok 5_err
```

Ticket scope narrows over time: - **File 1 (Aug 9 07:39-07:54):** tickets **1278201** (vol 0.3, 0.1) and **1278230** (vol 0.3, 0.5, 0.1) — both failing every attempt. - **File 2 (Aug 10 07:37-07:45):** only ticket **1278201** (240 mentions) — **1278230**'s legs were cancelled by then.

Accounts involved (fxsocket orderClose audit): - **edf84514-77c7-4326-93ab-2c6d20a84172** (trade **1278201**) - **d72087d3-5c47-4e23-9748-609f1444f116** (trade **1278230**)

DB state (prod, as of Aug 10): - Trade **ffcd18b3** (ticket **1278201**): **status=open** since Aug 7. Its partial legs: TP1 **fired** (0.5 lots, Aug 7 08:13), TP2 **pending** w/ **error_message='unknown ticket'**, TP3 **pending** w/ **error_message='unknown ticket'**. These two legs have been **stuck pending + retrying since Aug 7**. - Trade **ef03693f** (ticket **1278230**): **status=closed**; its 3 legs were cancelled Aug 10 07:13 with **parent trade not open**.

Root cause chain (code-verified): 1. The broker (FxSocket **OrderClose**) replies with **unknown ticket** — the position the worker thinks is open (ticket **1278201**) no longer exists / is unknown at the broker. 2. **partialTpMonitor.firePartial** (**worker/src/partialTpMonitor.ts:357**) classifies broker errors with the regex: **/not\s+found|already\s+closed|invalid\s+ticket|no\s+such\s+order/i** "**unknown ticket**" **does NOT match** this benign-reply regex. So instead of treating it as "position already gone" (which would cancel the leg and close the parent trade), it falls into the retry path (**partialTpMonitor.ts:385-408**): logs **fire failed**, rolls the leg back to **pending**, and retries — every ~400ms active tick, for days. 3. The parent trade stays **status='open'** in the DB (the reconcile monitors never mark it closed — see category 4), so the monitor keeps trying forever.

Fix needed: add **unknown ticket** (and ideally a broker retcode-based match) to the benign-error regex, or treat **FxsocketApiError** with the broker's "ticket not found" retcode as benign, so the leg is cancelled and the parent trade closed instead of retrying indefinitely.

2. **orderCloseAudit** — persist failure, null **signal_id** (file 1: 506; file 2: 120)

```
[orderCloseAudit] persist failed: null value in column "signal_id" of relation "trade_execution_logs" violates not-null constraint
```

Root cause (code-verified): **worker/src/orderCloseAudit.ts:28-43** (**registerOrderCloseAuditSupabase**) inserts into **trade_execution_logs** with **action**, **status**, **request_payload**, **error_message** — but **no signal_id**. The column is **NOT NULL** (**supabase/migrations/20260508190500_trade_execution_logs.sql:4**). Every order-close audit write fails at the DB and is only surface-logged.

Impact: the close-audit trail for ALL fxsocket order-close attempts in these windows is **not persisted**. This is independent of the **unknown ticket** issue (it would fail on successful closes too) but it means we lose the audit rows for every broker close.

Fix needed: either the audit insert must carry a **signal_id** (not always available at that call site — it is account/ticket-scoped), or **trade_execution_logs.signal_id** must become nullable for audit rows, or the audit should be written to a dedicated audit table/column that doesn't require **signal_id**.

3. **orderCloseAudit** — **source=fxsocket** close failures (file 1: 505; file 2: 120)

```
[orderCloseAudit] source=fxsocket account=<acc> ticket=1278201 volume=0.3 ok=false msg=unknown ticket
at ...fxsocketClient.js:989:51
at async PartialTpMonitor.firePartial (partialTpMonitor.js:239:28)
```

Same underlying cause as category 1 — these are the audit logs of the SAME failed partial closes (each **firePartial** attempt audits one failed **orderClose**). Stack confirms **PartialTpMonitor.firePartial** → **fxsocketClient.orderClose** → broker reject.

4. Reconcile monitors — empty broker snapshots, deferring close (file 1: 1,099; file 2: 396)

```
[openTradeReconcile] empty OpenedOrders with N tracked open trade(s) account=<uuid> — deferring ghost close (suspected disconnect)
[v2ReconcileMonitor] empty snapshot with N tracked legs broker=<uuid> anchor=<uuid> — deferring close (suspected disconnect)
```

- **File 1:** 525 **openTradeReconcile** + 574 **v2ReconcileMonitor** deferrals — accounts tracked 1–14 open trades each

while the broker returned **zero** open orders.

- **File 2:** 256 + 282 deferrals; additionally **90 fxsocketWsClient socket errors** appear here:

```
[fxsocketWsClient] socket error account=<uuid>: Client network socket disconnected before secure TLS connection was established
```

across 14 distinct accounts (5x each).

Code-verified safety behavior: both monitors deliberately refuse to mass-close off an empty snapshot (`openTradeReconcile.ts:52-61`, `engine/v2ReconcileMonitor.ts:322-329`). This is the correct guard — but combined with category 1 it means: the broker genuinely does not know ticket **1278201**, yet the DB keeps the trade open because reconcile defers on empty snapshots. The trade can never be reconciled closed and the partial monitor retries forever.

Observation: the empty snapshots are plausibly caused by the same TLS/connection instability seen in `fxsocketWsClient` (file 2), or by positions genuinely being closed at the broker. This needs a separate look at the FxSocket connection health across accounts.

5. `userListener` — Telegram poll / channel failures (file 2 is much worse)

Message	File 1	File 2
<code>poll getMessages failed user=c8a32918... channel=...: Request was unsuccessful 5 time(s)</code>	113	452
<code>ensureJoinedPublicChannel @gold_pro_forex_trader_xauusd ...: No user has "gold..."</code>	77	60
<code>ensureJoinedPublicChannel @gold_pro_trader_forex_signals0 ...: No user has "gold..."</code>	77	54
<code>ensureJoinedPublicChannel @forex_vipxauusd ...: No user has "forex_vipxauusd" as username</code>	80	48
<code>warmChannelEntity failed channel=a5c4ebd9...: Request was unsuccessful 5 time(s)</code>	36	17
<code>poll peer resolve failed user=d672de90... channel=a5c4ebd9...</code>	34	20

File 2 has **452 poll failures for one user (c8a32918) in 8 minutes** — message ingestion for that user is severely degraded. `ensureJoinedPublicChannel` cannot resolve 3 public channels (`@forex_vipxauusd`, `@gold_pro_forex_trader_xauusd`, `@gold_pro_trader_forex_signals0`) — it repeatedly fails to find a user with that username, suggesting the session cannot resolve those channel handles.

6. Telegram flood waits — background noise (file 1: 5,911; file 2: 6,671 un-prefixed)

```
[INFO] - [Sleeping for 31s on flood wait (Caused by messages.GetHistory)]  
[INFO] - [Sleeping for 30s on flood wait (Caused by messages.GetDialogs)]
```

Continuous Telegram rate-limiting (flood waits 24–31s) on `messages.GetHistory` / `messages.GetDialogs` throughout both windows — the underlying cause of the poll failures in category 5 and a constraint on message ingestion.

7. `tradeExecutor` — misc (file 2 only, 69 warn/error)

```
[tradeExecutor] apiForUuid: unknown broker uuid=<uuid> (4 error + 2)  
[tradeExecutor] /SymbolParams failed uuid=d72087d3... symbol=GBPUSD: SymbolSelect failed (27 warn)
```

- `apiForUuid: unknown broker` — the executor received a session uuid not in its `brokersById` map (broker not (re)loaded, or a stale uuid).
- `/SymbolParams ... SymbolSelect failed` — broker rejects symbol params for `GBPUSD` / `XAUUSD` on account `d72087d3`; matches the DB `signal_entry_pending_failed SymbolSelect failed` rows from the same morning.
- `[universalSignalParser] OpenAI failed: OpenAI timeout after 4000ms` (1x) — stage-2 parse fallback.

8. Risk-limit hit (file 1 only, 29x)

```
[copyLimitMonitor] limit hit broker=<uuid> channel=<uuid> equity=1875.65 breaches=risk:daily:...:mr-...cej flattened=false
```

A daily risk limit was hit on one broker/channel (equity \$1,875.65), no forced flatten.

9. Healthy signals (baseline, both files)

```
[telegram-conn] event=listener_healthy worker=a79d7a0282a4:12 user=<uuid> generation=1/2/3 connected=true  
[trailingStopMonitor] heartbeat rows=3 groups=0 (no SL updates this cycle)  
[autoManagementMonitor] heartbeat rows=35 groups=3 (no BE updates this cycle)  
[basketReconcileTargets] [effectiveStops] ... + drift sweep enqueued ...  
[virtualPendingMonitor] (file 2, 23 lines - healthy)
```

The listener and monitors report connected/healthy throughout; failures are concentrated in partial-TP firing, close auditing, FxSocket connection health, and Telegram transport.

10. Sentry business-issue errors (CSV export, 07:44-08:44 UTC Aug 10)

Errors 2026-08-10T08_44_34.csv — 35 captured business issues from `tscopier-worker-production` over exactly one hour (07:44:37 → 08:44:00 UTC). All are `captureBusinessIssue` events (distinct from the console-log trace items above).

Count	Event (error code)	Message
12	<code>trade_copy_failed</code>	Signal accepted but trade copy permanently failed
6	<code>trade_copy_blocked</code>	Trade copy skipped before broker send
5	<code>broker_order_rejected</code>	Broker rejected trade copy order
4	<code>trade_copy_blocked</code>	Signal dispatch skipped before trade execution
4	<code>broker_account_unavailable</code>	Signal dispatch skipped before trade execution
4	<code>trade_management_failed</code>	Trade management instruction skipped or failed before completion

Code-level meaning of each code:

- `trade_copy_failed` (12x)** — emitted from two places:
 - `orderLegExecution.ts:426` — broker `OrderSend` permanently rejected with reason `INSUFFICIENT_MARGIN` (matches the “Not enough money” failures seen in the DB window: XAUUSD sells at 04:03/05:53/05:56 UTC).
 - `dispatch.ts:942` — an entry signal where **no** broker leg opened after retries (`entryFailureReason`); user exposure may be partial/absent.
- `trade_copy_blocked` “...before broker send” (6x)** — `managementExecutor.ts:410-414` : trade-copy skipped pre-send, reason anything **other than** `broker_session_not_connected` (that maps to `broker_account_unavailable`).
- `trade_copy_blocked` “Signal dispatch skipped before trade execution” (4x)** — `dispatch.ts:340-344` : dispatch-level skip with a `copy_limit / risk / max_` reason (daily risk limit — consistent with the `[copyLimitMonitor] limit hit ... breaches=risk:daily` line in file 1).
- `broker_order_rejected` (5x)** — `orderLegExecution.ts:429` : `OrderSend` rejected with any reason code that isn't `INSUFFICIENT_MARGIN` or `SYMBOL_UNSUPPORTED` (default `BROKER_ORDER_REJECTED` , e.g. `MARKET_CLOSED` / `INVALID_LOT` / `BROKER_RATE_LIMITED`).
- `broker_account_unavailable` (4x)** — two sources:
 - `dispatch.ts:343 / :755` : dispatch skip because the signal's configured broker is `broker_session_not_connected` / no eligible broker account.
 - `managementExecutor.ts:413` : mgmt pre-send skip with `broker_session_not_connected` . These line up with the FxSocket TLS disconnects + empty snapshots in category 4 (a broker session considered unavailable → copies skipped).
- `trade_management_failed` (4x)** — `managementExecutor.ts:479-494` : a management instruction (modify/BE/SL/TP) was skipped or failed before completing (`reasonCode` like `no_open_trade` / `none`).

Relation to the DB/execution-log window (Aug 10 00:00-08:00 UTC): this hour's 35 issues match the same failure families seen all morning — margin exhaustion (`trade_copy_failed` ↔ `order_send Not enough money`), broker session/account unavailability (`broker_account_unavailable` ↔ empty snapshots / TLS disconnects), risk-limit gating (`trade_copy_blocked` ↔ `copyLimitMonitor` limit hits), and broker rejections (`broker_order_rejected` ↔ `SymbolSelect`).

failed / rejected orders). The CSV is the tail end of a degraded window that ran from ~00:00 UTC.

DB cross-reference

- Sentry file-1 window (Aug 9 07:39–07:54 UTC): **0 rows** in `trade_execution_logs` — consistent with category 2 (every audit insert failed at the DB).
- Sentry file-2 window (Aug 10 07:37–07:45 UTC): no `partial_tp_fired` rows landed in that exact window (the monitor's own `partial_tp_fired` failed inserts at `partialTpMonitor.ts:393` include `signal_id` so they persist) — the DB rows for the same ticket pattern appear at **08:11–08:41 UTC** (515 rows, `partial_tp_fired` / `failed` / `unknown ticket`). The sentry export is a sampled/subset slice; the underlying retry loop is the same across all these windows.
- `trades` (prod): trade `1278201` still `status=open`; trade `1278230` `status=closed` with legs cancelled.
- `partial_tp_legs` (prod): legs `297bb64e` (TP2) and `bdef006d` (TP3) for trade `1278201` still `pending` with `error_message='unknown ticket'`.
- **CSV window (Aug 10 07:44–08:44 UTC)**, `trade_execution_logs`: 397 `partial_tp_fired` / `failed` / `unknown ticket`, 47 `order_send success`, 10 `order_send failed` / `HTTP 500`, 10 `order_send skipped`, 3 `signal_entry_pending_failed` / `SymbolSelect failed`, 6 `partial_tp_fired success`, plus risk-limit and channel-filter dispatch skips. The 35 CSV business issues sit on top of (and are explained by) these DB rows — the pipeline was heavily degraded through this hour.

Conclusions

1. **Partial-TP is broken for ticket `1278201`** (and was broken for `1278230` until its legs were cancelled). The broker does not know the ticket; the worker's benign-error classification regex does not recognize `unknown ticket`, so it retries forever instead of cancelling the leg and closing the parent trade. **This is the dominant production failure (505 errors in 14.5 min).**
2. **Order-close auditing never persists** — `signal_id` is omitted from the audit insert against a `NOT NULL` column. Close-audit data is being silently dropped in production.
3. **FxSocket connection instability** (`fxsocketWsClient` TLS disconnects, empty snapshots across 14+ accounts) plausibly caused the reconcile monitors to defer ghost closes — correct safety behavior, but it leaves stale `open` trades (like `1278201`) un-reconciled, feeding the retry loop in (1).
4. **Telegram transport degraded** on the affected session(s): persistent flood-waits + 452 poll failures for user `c8a32918` in 8 min + unresolvable public-channel usernames.
5. **Secondary issues:** `SymbolSelect failed` on `d72087d3` (blocks entries), `apiForUuid: unknown broker`, and one OpenAI stage-2 timeout.
6. **A broad trading-impact window, not just partial-TP:** 35 captured business issues in the 07:44–08:44 UTC hour alone — 12 copy failures (margin/entry), 10 copies blocked (risk-limit + pre-send), 5 broker order rejections, 4 broker-account-unavailable, 4 management failures. This confirms the whole copy pipeline was degraded through Aug 10 morning, consistent with the DB `order_send` / `signal_entry_pending_failed` failures.

Next steps

1. **Fix the benign-error regex** in `partialTpMonitor.ts:357` (add `unknown ticket` / broker retcode match) so a gone position cancels the leg + closes the parent instead of retrying forever.
2. **Fix the order-close audit persistence** (`orderCloseAudit.ts:28-43`) — supply `signal_id` or make the audit write not depend on a `NOT NULL` `signal_id`.
3. **Manually reconcile the stale trade `1278201`** (`ffcd18b3`) — verify at the broker whether the position truly is gone; if so, close the DB row + cancel the two stuck partial legs.
4. **Investigate FxSocket connection health** across the 14+ affected accounts (TLS disconnects → empty snapshots → `broker_account_unavailable` skips).
5. **Investigate the Telegram session** for user `c8a32918` and the 3 public channels (flood waits + `ensureJoinedPublicChannel` failures).
6. **Investigate margin exhaustion** causing the repeated `trade_copy_failed` / `Not enough money` on XAUUSD sells — whether failed partial closes (category 1) left positions over-leveraged and drained available margin.